

PROYECTO 1
Módulo: M0616- Proyecto de desarrollo web
Profesor: Albert Guardiola
Fecha límite: ENTREGA1 – 23/9/26 ENTREGA2 – 30/9/26 ENTREGA3 – 14/10/26
Método de entrega: A través del Clickedu de la asignatura.

PROYECTO 1 - Automatización, integración continua y despliegue de un proyecto web

PARTE A. CONTEXTO Y EXPLICACIÓN GENERAL DEL PROYECTO

A.1. Contexto

En un proyecto de desarrollo web profesional, GitHub no se utiliza únicamente para guardar el código fuente.

Un repositorio puede incorporar un conjunto de procesos automatizados que permiten:

- trabajar con ramas y Pull Requests;
- revisar los cambios antes de incorporarlos;
- ejecutar tests automáticamente;
- detectar errores antes de hacer un merge;
- gestionar las dependencias del proyecto;
- analizar posibles vulnerabilidades;
- desplegar automáticamente una aplicación.

En esta práctica asumirás el rol de una persona desarrolladora que debe convertir un proyecto web convencional en un proyecto con un **flujo de trabajo DevOps automatizado mediante GitHub**.

Partirás de una aplicación web desarrollada con **HTML, CSS y JavaScript** y tendrás que ir incorporando progresivamente diferentes mecanismos de automatización.

A.2. Proyecto web de partida

El proyecto de partida es **DevTasks**, una aplicación web sencilla de gestión de tareas.

La aplicación permite:

- crear tareas;
- marcarlas como completadas;
- eliminarlas;
- filtrarlas;
- consultar estadísticas;
- conservar los datos mediante `localStorage`.

El proyecto está desarrollado con:

- HTML
- CSS
- JavaScript
- npm
- Vitest

Estructura inicial

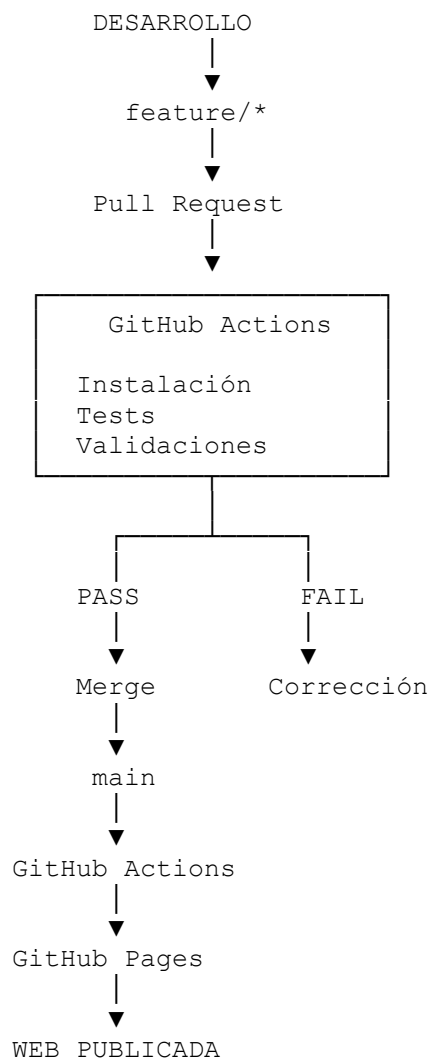
```
devtasks/  
├── index.html  
├── css/  
│   └── style.css  
├── js/  
│   └── app.js  
├── tests/  
│   └── app.test.js  
├── package.json  
├── .gitignore  
└── README.md
```

El proyecto inicial contiene también algunas funciones JavaScript susceptibles de ser separadas y testeadas.

A.3. Objetivo general

El objetivo de la práctica es **transformar el proyecto inicial en un repositorio GitHub con un flujo de desarrollo y despliegue automatizado**.

Al final de la práctica deberás llegar a una arquitectura conceptual parecida a:



Además, el repositorio deberá incorporar mecanismos de gestión de dependencias y, opcionalmente, mecanismos de análisis de seguridad.

A.4. GitHub Skills

Antes de empezar la configuración DevOps del proyecto deberás completar los tutoriales de GitHub Skills indicados a continuación.

Skills obligatorios

Bloque 1 — GitHub y colaboración

1. **Introduction to GitHub**
2. **Review pull requests**
3. **Resolve merge conflicts**
4. **GitHub Pages**

Bloque 2 — Automatización

4. **Hello GitHub Actions**
5. **Test with Actions**
6. **Secure your repository's supply chain**

Skill de ampliación

7. **Introduction to CodeQL**

El curso de CodeQL es opcional y forma parte de las actividades de ampliación.

A.5. Relación entre los Skills y el proyecto

Los tutoriales no son una actividad independiente.

Cada Skill deberá ser aplicado posteriormente al proyecto web.

GitHub Skill	Conocimiento adquirido	Aplicación al proyecto
Introduction to GitHub	Repositorios, commits, ramas	Crear y organizar el repositorio
Review pull requests	Pull Requests y revisión	Flujo de trabajo basado en PR
GitHub Pages	Publicación web	Despliegue de la aplicación
Hello GitHub Actions	Workflows, jobs y steps	Crear los pipelines
Test with Actions	CI y tests	Ejecutar tests automáticamente
Secure repository supply chain	Dependencias y Dependabot	Automatizar actualizaciones
Introduction to CodeQL	Code Scanning	Análisis de seguridad — ampliación

PARTE B. ENTREGAS 1 Y 2 — EVIDENCIAS GITHUB SKILLS

La primera parte de la práctica consiste en demostrar que has realizado los tutoriales de GitHub Skills.

Las evidencias se dividirán en dos entregas.

ENTREGA 1 — Skills de GitHub colaborativo

Tutorials

Deberás completar:

- Introduction to GitHub
- Review pull requests
- Resolve merge conflicts
- GitHub Pages

Evidencias

Entrega un documento PDF que contenga:

2. Evidencia de finalización

Por cada tutorial deberás incluir capturas de pantalla que permitan comprobar que la actividad ha sido completada.

3. Reflexión

Por cada Skill deberás explicar brevemente:

- qué has aprendido;
- qué problema resuelve;
- cómo lo aplicarás al proyecto DevTasks.

No se trata únicamente de explicar qué hace la herramienta. Debes relacionar el conocimiento con el proyecto.

Ejemplo

SKILL: Review Pull Requests

He aprendido a crear Pull Requests y revisar los cambios antes de incorporarlos a la rama principal.

APLICACIÓN AL PROYECTO:

Utilizaré Pull Requests para incorporar las modificaciones realizadas en ramas feature/* a main.

ENTREGA 2 — Skills DevOps

Tutorials

Deberás completar:

- Hello GitHub Actions
- Test with Actions
- Secure your repository's supply chain

Evidencias

Entrega un segundo documento PDF con:

1. Evidencias de finalización

Capturas de pantalla de los tutorials completados.

2. Skill adquirido

Por cada tutorial explica:

- qué has aprendido;
 - qué problema resuelve;
 - qué elemento del proyecto deberás configurar gracias a este conocimiento.
-

PARTE C. APLICACIÓN DE LOS SKILLS AL PROYECTO WEB

A partir de este punto deberás utilizar los conocimientos adquiridos en los tutoriales para transformar el proyecto DevTasks.

La implementación se debe realizar **paso a paso y en el orden indicado**.

PASO 1. Crear el repositorio del proyecto

Crea un repositorio nuevo en GitHub para el proyecto.

El repositorio deberá tener:

- un nombre identificativo;
- un `README.md`;
- una rama principal `main`;
- el proyecto web de partida.

!!!ATENCIÓN!!! La app de partida tiene un bug de código. Podéis comprobarlo desplegando la app en local con LiveServer y creando una tarea: veréis que la tarea no se crea correctamente. Encontrad el bug en el código y corregidlo.

Pista: comparad la declaración de la función `createTask` con su llamada...

Evidencia

Deberás poder demostrar:

- existencia del repositorio;
- estructura de archivos;
- historial de commits.

PASO 2. Preparar el proyecto JavaScript

Antes de empezar con GitHub Actions, **prepara el proyecto para poder ejecutar tests localmente**.

1. **Instala NodeJs** en tu ordenador (<https://nodejs.org/en>) Lo necesitaremos para usar el gestor de paquetes `npm`, que viene incluido en la instalación de NodeJs.
2. Después de instalar NodeJs instala las dependencias del proyecto descritas en el archivo `Package.json`. Hazlo con el comando (en la raíz del proyecto):

```
npm install
```

3. En la carpeta *test/* hay un archivo *.test.js* que tiene definida una suite de tests de algunas funciones clave de la aplicación. Comprueba que los tests se pueden ejecutar con:

```
npm test
```

Identifica qué funciones se testean y cuál es el resultado del test.

PASO 3. Refactorizar el JavaScript

Antes de automatizar los tests deberás **separar la lógica de negocio de la manipulación del DOM**.

La versión inicial contiene toda la funcionalidad en:

```
js/app.js
```

Deberás refactorizarla.

El objetivo es llegar, como mínimo, a:

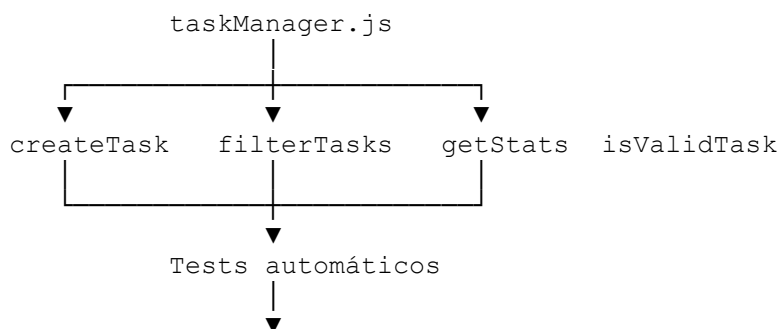
```
js/  
├── app.js  
└── taskManager.js
```

taskManager.js

Deberá contener la lógica que pueda ser ejecutada y testeada independientemente del navegador. Hay que mover a *taskManager.js* las funciones puramente de lógica de negocio, que son independientes del DOM. Esto hará que:

- Se puedan **testear estas funciones sin error**.
- Se **separen las responsabilidades** de cada módulo: en *app.js* quedará todo lo que tiene que ver con la interfaz gráfica y en *taskManager.js* todo lo que tiene que ver con cálculos y comprobaciones independientes de la interfaz y – por tanto – reutilizables en otros contextos.

El objetivo es conseguir:



PASO 4. Crear y ampliar los tests

Crea tests para comprobar el comportamiento de las funciones de *taskManager.js*. Ten en cuenta que, después de la refactorización, el archivo de tests deberá incluir, además de las funciones necesarias para el test:

```
import { describe, it, expect } from 'vitest';
```

las importaciones de las funciones de *taskManager.js* a testear:

```
import { isValidTask, createTask, filterTasks, getTaskStats } from  
'../js/taskManager.js';
```

Añade un test nuevo para cada función. **En clase explicaremos la arquitectura que tienen las funciones de test:** describe (it, expect, assert).

Ejecuta los tests localmente:

```
npm test
```

Antes de continuar, todos los tests deben pasar.

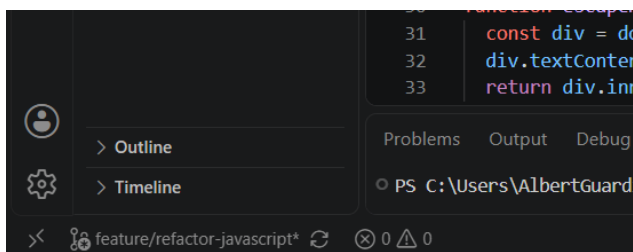
En este momento todavía estás ejecutando los tests localmente. Posteriormente aprenderás a hacer que GitHub los ejecute automáticamente.

PASO 5. Crear una rama de funcionalidad

A partir de este momento no deberás trabajar directamente sobre `main`.

Crea una rama `feature/refactor-javascript` (puedes hacerlo desde tu cuenta de GitHub.com), por ejemplo:

En VSCODE, asegúrate de que cambias de rama.



Sube a esta rama el estado actual del proyecto (bug corregido y tests añadidos).

PASO 6. Crear una Pull Request

Cuando hayas terminado la modificación:

1. Sube la rama a GitHub.
2. Crea una Pull Request hacia `main`.
3. Describe los cambios realizados.
4. Revisa los archivos modificados.

La Pull Request será el punto de entrada de nuestro proceso de Integración Continua.

PASO 7 y 8. Crear el primer workflow de GitHub Actions

Ahora deberás automatizar una tarea que hasta ahora hacías manualmente.

Crea:

```
.github/  
└─ workflows/  
    └─ ci.yml
```

El workflow deberá:

1. ejecutarse cuando se haga una Pull Request hacia `main`;
2. obtener el código del repositorio;
3. preparar Node.js;
4. instalar las dependencias;
5. ejecutar los tests.

El archivo YAML debería ser algo parecido a esto:

```
name: Continuous Integration (CI)  
  
on:  
  pull_request:  
    branches:  
      - main  
  
jobs:  
  build-and-test:  
    name: Install Dependencies and Run Tests  
    runs-on: ubuntu-latest  
  
    steps:  
      - name: Checkout repository  
        uses: actions/checkout@v4  
  
      - name: Setup Node.js environment  
        uses: actions/setup-node@v4  
        with:  
          node-version: '20'  
          cache: 'npm'  
  
      - name: Install dependencies  
        run: npm ci  
  
      - name: Execute unit tests  
        run: npm test
```

Asegúrate de subir los cambios al repo de GitHub. El

resultado deberá ser visible en GitHub Actions.

PASO 9. Provocar un error deliberadamente

Para demostrar que entiendes el funcionamiento del pipeline, deberás provocar un error controlado.

Por ejemplo:

1. Modifica temporalmente una función.
2. Crea una Pull Request.
3. Comprueba que GitHub Actions detecta el problema.
4. Observa que los tests fallan.
5. Corrige el problema.
6. Vuelve a ejecutar el pipeline.
7. Comprueba que los tests vuelven a pasar.

Evidencia obligatoria

Deberás capturar:

```
PR
↓
GitHub Actions
↓
TEST FAILED
```

y posteriormente, una vez arregles el bug y vuelvas a subir el código:

```
PR
↓
GitHub Actions
↓
TEST PASSED
```

Esta será una de las evidencias más importantes de la práctica.

PASO 10. Proteger la rama `main`

Configura el repositorio para que `main` no pueda recibir directamente los cambios sin pasar por el proceso establecido.

Como mínimo deberás conseguir:

- Pull Request obligatoria;
- workflow de CI obligatorio;
- no permitir hacer merge si el CI falla.

El objetivo es implementar:

CI PASS

↓

MERGE PERMITIDO

CI FAIL

↓

MERGE BLOQUEADO

Paso a paso para la configuración en GitHub

1. En tu repositorio de GitHub, ve a la pestaña superior **Settings**.
2. En el menú lateral izquierdo, selecciona **Branches** (o **Rulesets** según la interfaz de tu organización).
3. En la sección **Branch protection rules**, haz clic en el botón **Add branch ruleset** o **Add rule**.
4. En **Branch name pattern**, escribe exactamente:

Plaintext

main

5. Marca las siguientes casillas de protección obligatorias:
 - ☒ **Require a pull request before merging** (Exige una PR antes de poder incorporar cambios).
 - ☒ **Require status checks to pass before merging** (Exige que las comprobaciones de la CI hayan pasado).
6. En la barra de búsqueda de status checks que se abrirá debajo, busca y selecciona el nombre del job de tu workflow:

Plaintext

Install Dependencies and Run Tests

7. (Opcional) Marca la casilla **Do not allow bypassing the above settings** para aplicar la regla a los administradores del repositorio.
8. Haz clic en **Create** o **Save changes** al final de la página.

PASO 11. Configurar GitHub Pages

Publica la aplicación mediante GitHub Pages.

La aplicación deberá poder consultarse mediante una URL pública.

Comprueba que:

- la página inicial funciona;
- el CSS se carga;
- JavaScript funciona;
- las funcionalidades de la aplicación son operativas.

PASO 12. Automatizar el despliegue

No queremos tener que publicar manualmente la aplicación si la PR se resuelve con un MERGE.

Paso 1. Activar GitHub Pages en el repositorio

1. Abre el repositorio en GitHub y ve a la pestaña **Settings** (Configuración).
2. En el menú lateral izquierdo, entra en la sección **Pages** (dentro de *Code and automation*).
3. En la sección **Build and deployment > Source**, cambia la opción por defecto (*Deploy from a branch*) y selecciona **GitHub Actions**.

Paso 2. Crear el workflow `.github/workflows/deploy.yml`

En tu entorno local (o desde una nueva rama de trabajo), crea el archivo de workflow para el despliegue:

```
.github/  
├── workflows/  
│   └── ci.yml  
└── deploy.yml
```

El contenido de `deploy.yml`:

```
name: Deploy Web App to GitHub Pages  
  
# Trigger: Se ejecuta solo cuando se hace un PUSH o MERGE directamente en la rama  
main  
on:  
  push:  
    branches:  
      - main  
  
# Permisos necesarios para poder escribir y desplegar en GitHub Pages  
permissions:  
  contents: read  
  pages: write  
  id-token: write  
  
# Garantiza que solo haya un despliegue activo a la vez  
concurrency:  
  group: 'pages'  
  cancel-in-progress: true
```

```
jobs:
  deploy:
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    runs-on: ubuntu-latest

    steps:
      # Step 1: Descarga el código de la rama main
      - name: Checkout repository
        uses: actions/checkout@v4

      # Step 2: Configura el entorno de Pages
      - name: Setup Pages
        uses: actions/configure-pages@v4

      # Step 3: Empaqueta y sube los archivos estáticos (.html, .css, .js) como
      artefacto
      - name: Upload artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: '.' # Publica todo el directorio raíz

      # Step 4: Despliega el artefacto en el entorno oficial de GitHub Pages
      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

Este workflow deberá permitir desplegar automáticamente la versión de `main` en GitHub Pages.

Paso 3. Ejecutar y verificar el despliegue

1. Haz commit y push de este archivo **en la rama main** (o hazlo mediante una PR y acepta el *Merge* a main).
2. Ve a la pestaña **Actions** de GitHub y observa el workflow **Deploy Web App to GitHub Pages**.
3. Una vez finalice en verde, entra en **Settings > Pages** o en el enlace generado en la página principal del repositorio.
4. Abriremos la URL pública (tipo `[https://usuari.github.io/nom-repositorio/](https://usuari.github.io/nom-repositorio/)`).
5. Comprueba manualmente que la página se ve bien, el CSS se carga, puedes añadir/filtrar tareas y el `localStorage` mantiene los datos.

El flujo final será:

```
Pull Request
↓
CI
```

```
↓  
Tests  
↓  
PASS  
↓  
Merge  
↓  
main  
↓  
Deploy  
↓  
GitHub Pages
```

PASO 13. Gestionar las dependencias

El proyecto utiliza dependencias npm.

Configura Dependabot para que compruebe periódicamente las actualizaciones de las dependencias.

El repositorio deberá incluir:

```
.github/  
└─ dependabot.yml
```

La configuración deberá contemplar las dependencias del proyecto y las dependencias de los GitHub Actions utilizados, cuando sea aplicable.

PASO 14. Comprobar Dependabot

Comprueba que la configuración de Dependabot es visible y correcta.

Si durante el periodo de trabajo Dependabot genera una actualización, revísala y documenta el proceso.

Si no se genera ninguna actualización durante la práctica, será suficiente demostrar correctamente la configuración.

PASO 15. Actualizar el README

El README deberá documentar el proyecto y su sistema DevOps.

Debe incluir, como mínimo:

Descripción

Qué hace DevTasks.

Instalación

Cómo instalar las dependencias.

Tests

Cómo ejecutar los tests.

GitHub Actions

Qué workflows existen y qué hace cada uno.

Pull Requests

Qué flujo de trabajo se ha establecido.

Deploy

Enlace a la web publicada.

Dependencias

Cómo se gestionan las actualizaciones.

Arquitectura

Explicación de la separación entre:

```
app.js
taskManager.js
tests/
```

PASO 16. Comprobar el flujo DevOps completo

Antes de la entrega final deberás demostrar el funcionamiento completo del sistema.

Realiza una pequeña modificación en el proyecto.

El proceso deberá ser:

1. Crear rama
- ↓
2. Modificar código
- ↓

```
3. Commit
   ↓
4. Push
   ↓
5. Pull Request
   ↓
6. GitHub Actions
   ↓
7. Tests
   ↓
8. PASS
   ↓
9. Merge
   ↓
10. Deploy automático
   ↓
11. GitHub Pages actualizado
```

PASO 17. Actividad de ampliación: CodeQL

Completa el tutorial:

Introduction to CodeQL

e incorpora Code Scanning al proyecto.

El objetivo es que el repositorio pueda detectar automáticamente determinados patrones de vulnerabilidades en el código.

Skill aplicado

Introduction to CodeQL

Esta actividad es opcional.

ENTREGA 3 — REPOSITORIO DEVOPS Y EVIDENCIAS

La entrega final deberá incluir:

1. Repositorio GitHub

Enlace al repositorio final.

El repositorio deberá ser accesible para el profesorado.

2. Aplicación publicada

Enlace a la versión publicada con GitHub Pages.

3. Evidencias

Entrega un documento PDF con capturas que demuestren:

Git

- repositorio;
- ramas;
- commits.

Pull Request

- una PR real;
- revisión;
- merge.

CI

- ejecución correcta;
- tests superados.

Error

- ejecución con tests fallidos;
- corrección posterior.

Branch protection

- configuración de protección de `main`.

Deploy

- ejecución del workflow;
- aplicación publicada.

Dependabot

- configuración;
- actualización/PR si se ha generado alguna.

Aplicación

- web publicada y funcionando.

Opcional

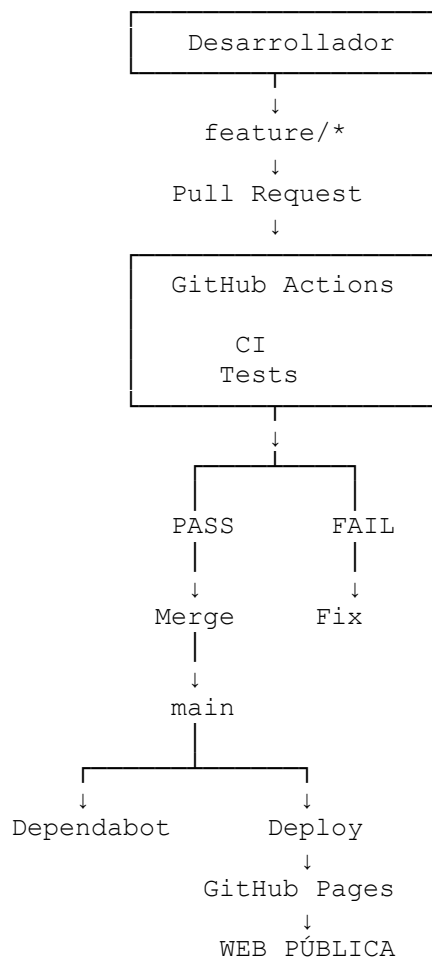
- CodeQL / Code Scanning.
-

RESULTADO FINAL

Al final de la práctica habrás transformado un proyecto web convencional:

HTML
CSS
JavaScript

en un proyecto con un flujo DevOps:



El producto final no es solo la aplicación DevTasks.

El producto final es el proyecto web + el repositorio + el flujo de trabajo automatizado + las evidencias que demuestran que funciona.